



**TECHNICAL QUESTIONS
LOMBA KOMPETENSI SISWA (LKS)
SMK KABUPATEN MALANG
2025**

**Cloud Computing
26 FEBRUARI 2025
SMKN 2 TUREN**

DESCRIPTION OF PROJECT AND TASKS

This module is seven hours: Deploying a CMS application.

The goal of this Project is to automatically deploy and scale the back end of a CMS application. Today you will deploy a highly available, scalable, and efficient web application using the Apache web server.

You must also build the required initial support services. This CMS application has service dependencies as outlined in the Technical Details section below.

This application responds well to both caching and horizontal scaling. It will not work “out of the box.” Instead, you will have seven hours to launch the infrastructure and deploy the application.

TASK

1. Log into the AWS console.
2. Switch Region to Asia Pacific (Singapore)
3. Set up the Instance EC2; you can read Technical Details
4. Set up the Database; you can read SQL Database details.
5. Set up the Cache service; you can read Redis Cache details.
6. Setup your Auto Scaling and Load Balancer from Launch Template; you can read Auto Scaling and Load Balancer details.
7. Set up the Bucket; you can read Object Storage (S3) details.
8. Set up your VPC Configuration. You can read the details in the Architecture Details section.
9. Set up Cloud Front for your CDN, connecting to your S3 bucket. serving your assets
10. Set up Cloud Watch for monitoring your server

TECHNICAL DETAILS

1. The CMS application has source codes that you must install from the original source to your webserver.
2. User Region Asia Pacific (Singapore)
3. Backend Server must be deployed on **Debian** Operating System
4. Backend Server must be deployed on t3.medium instance type
5. You must can login using SSH on your server
6. Install Application; You can read Application Details
7. This app requires a connection to a Redis database and a MySQL database. As with the previous deployment, you can set up a Redis server and a database server. The more efficient you run the infrastructure, the faster your servers will respond to requests and the more points you will earn.
8. This app will store the assets data like pictures in S3 Bucket. More details for the S3 instructions at S3 Service Details section.

APPLICATION DETAIL

The application was created using Laravel Framework. The application gets data from the database and cache the data on redis to speed up the load page.

PRE-REQUIREMENT

Make sure your instance is connected to the internet for running all of these instructions.

You need install these package on your server backend server :

- Apache 2.4.56
- PHP 8.2
- Node v20.13.1
- Npm 10.5.2
- Composer

PHP Module : php fpm, curl, xml, mbstring, gd, imagick, imap, intl, mysqli, pgsql, zip

APPLICATION SETUP

This Application based on Laravel Framework,

Go to folder /var/www/html/

1) Run this command to install Laravel Framework

- sudo chown <USER_INSTANCE_EC2> /var/www/html
- composer global require laravel/installer
- export PATH="\$PATH:\$HOME/.config/composer/vendor/bin"
- laravel new lks2025
- cd lks2025
- change value on composer.json
 - before : "laravel/framework": "^12.0",
 - after : "laravel/framework": "^11.0",
- rm -f composer.lock
- chmod -R 777 storage/ bootstrap/cache/
- npm install && npm run build
- composer run dev
- You must prepare the Database (Task 4) and Redis Cache (Task 5) before continuing with this step
- Copy .env.example to .env file in the php source code folder. Change variable below.

```

-----.ENV-----
APP_ENV=local
APP_URL=<YOUR_IP_ADDRESS>

DB_HOST= <YOUR_DATABASE_IP>
DB_PORT= <YOUR_DATABASE_POST>
DB_DATABASE= <YOUR_DATABASE_NAME>
DB_USERNAME= <YOUR_DATABASE_USERNAME>
DB_PASSWORD= <YOUR_DATABASE_PASSWORD>

CACHE_DRIVER= phpredis

REDIS_HOST= <YOUR_REDIS_IP>
REDIS_PASSWORD=
REDIS_PORT=

-----.ENV-----

```

2) Run this command to migrate the database

- php artisan migrate
- php artisan storage:link
- php artisan key:generate

3) Run this command to install Filament Framework

- composer require filament/filament:"^3.2" -W
- php artisan filament:install --panels (admin)
- php artisan make:filament-user (admin, admin@lks2025.com, lks2025)
- php artisan filament:optimize

4) Modify apache webserver configuration

- Change Document root to : /var/www/html/lks2025/public
- Enable these module rewrite, proxy fcgi, setenvif, php fpm

If all is done you can access the client apps with http://YOUR_IP_ADDRESS and http://YOUR_IP_ADDRESS/admin in the web browser.

Test login with your credentials that you've created before.

AUTOSCALING + LOAD BALANCER

Create a Launch Template from the Instance you created

Build Auto Scaling

Create a Load Balancer, then connect to your autoscaling.

If all is done you can access the client apps with `http://LOAD_BALANCER` in the web browser.

SQL DATABASE

you need to deploy a centralized relational database management system (RDMS). You need to use a reliable Relational Database Service (RDS) with a serverless concept. Choose MySQL 8.0.40, You can use the free tier first. Create a database with the name “**lks2025**”. Once your serverless already provisioned, you will need to create the table necessary to serve the request. If you can not login to mysql, try to install the default mysql client.

OBJECT STORAGE (S3)

This application should use object storage to store user asset data such as pictures and documents.

Create a Bucket.

Copy all files on this folder to your bucket : `/var/www/html/lks2025/public/build`

View your assets url with bucket url.

Note: The aws bucket name must be unique. Use this bucket name format i.e.

`lks2025-your_name-your-city_province`

Ex: `lks2025_adi_cahyono_kab_malang_jatim`

CLOUD FRONT

Setup Cloud Front to connect on your S3.

View your assets url with Cloud Front URL

REDIS CACHE

You will want to deploy a centralized Redis OSS Cache Service for the same reasons as you want to create a centralized database service, for efficiency. Just as with databases, you can create a per-instance deployment of Redis but that will operate slower (fewer responses to requests) than using a centralized solution. This redis workload will be quite intensive. The app will store any API data temporarily in redis and will be used frequently when app requests come.

ARCHITECTURE DETAILS

